SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14 COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:100

AnnexureA

EXPERIMENTS

Code of Conduct:

I Santhosh kumar .v /192411015(Name / Reg No) certify that this submission is my original work and

that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: V.Santhosh Kumar

AnnexureA

EXPERIMENTS

Q1.Youaredevelopingacompiler-baseddeskcalculatorthatevaluatesarithmeticexpressions usingsyntax-directeddefinitions(SDD).

Thesystemshould:

(i)Acceptanarithmeticexpressionwithoperators+,-,*,/andparentheses

(ii)Usesyntax-directedevaluationtocomputevaluesbasedonoperatorprecedence

(iii)Optimizeevaluationbyreducingunnecessarytemporarycomputations

(iv)Displaythefinalresultoftheexpression

DemonstratehowSDDrules(synthesizedattributes)areusedforexpressionevaluation.

SDD

Grammarwithsemanticrules:

E → E + T { E.val = E1.val + T.val }

E → E - T { E.val = E1.val - T.val }

E → T { E.val = T.val }

T → T * F { T.val = T1.val * F.val }

T → T / F { T.val = T1.val / F.val }

T → F { T.val = F.val }

F → (E) { F.val = E.val }

F → digit { F.val = digit }

Q2.Inacompiler,postfixexpressionsareevaluatedduringcodegenerationusingstack-based bottom-upevaluation.

WriteaCprogramthat:

(i)Evaluatesapostfixexpression

(ii)DemonstratesS-attributeddefinitionevaluation

(iii)Showshowintermediateresultsarecomputed

Q3.YouaredesigningaparserthatusesL-attributeddefinitions,whereinheritedattributesare passedfromparenttochild.

WriteaCprogramthat:

(i).Evaluatesasimpleexpressionlikea + b * c

(ii)Usesfunctionparameterstosimulateinheritedattributes

(iii).Ensurescorrectprecedencehandling

(iv).Demonstratestop-downevaluationlogic

Explainhowinheritedattributesinfluencecomputation.

Q4.Inacompiler,itisimportanttocheckwhethertwotypeexpressionsareequivalent.

WriteaCprogramthat:

(i).Acceptstwotypeexpressions(e.g.,int,float,int*)

(ii).Comparesthemfortype equivalence

(iii).Displayswhethertheyare:

(a).Equivalent

(b).Notequivalent

Extendthelogictohandlepointertypes(int*,float*).

Q5.Youarebuildingasemanticanalyzerthatdetectstypeerrorsinarithmeticexpressions.

WriteaCprogramthat:

(i).Acceptstwooperandsandtheirtypes

(ii).Checkswhethertheoperation(e.g.,+,*)isvalid

(iii).Reports:

(a).Validexpression

(b).Typeerror(e.g.,int + char*)

Demonstratehowcompilersdetectsemanticerrorsduringtypechecking

Answers:

1) calculator:

```
#include<stdio.h>
#include<ctype.h>
charexpr[100];
intpos=0;
intE();
intT();
intF();
intE()
{
intval=T();
while(expr[pos]=='+'||expr[pos]=='-')
{
charop=expr[pos++];
intvalue2=T();
if(op=='+')
val=val+value2;
else
val=val-value2;
}
returnval;
}
intT()
{
intval=F();
```

```
while(expr[pos]=='*'||expr[pos]=='/')
{
charop=expr[pos++];
intvalue2=F();
if(op=='*')
val=val*value2;
else
val=val/value2;
}
```

```
returnval;
}
intF()
{
intval;
if(expr[pos]=='(')
{
pos++;
val=E();
pos++;
}
else
{
val=expr[pos]-'0';
pos++;
}
returnval;
}
intmain()
{
printf("Enterarithmeticexpression:");
scanf("%s",expr);
printf("Result=%d\n",E());
return0;
}
```

Output:

2)Postfix Evaluation

```
#include<stdio.h>

#include<ctype.h>

intstack[100];

inttop=-1;
```

```
voidpush(intvalue)
{
stack[++top]=value;
}
intpop()
{
returnstack[top--];
}
intmain()
{
charpostfix[100];
inti,a,b,result;
printf("Enterpostfixexpression:");
scanf("%s",postfix);
for(i=0;postfix[i]!='\0';i++)
{
if(isdigit(postfix[i]))
{
push(postfix[i]-'0');
}
else
{
b=pop();
a=pop();
switch(postfix[i])
{
```

```
case'+':
result=a+b;
break;
case'-':
result=a-b;
break;
case'*':
result=a*b;
break;
case'/':
result=a/b;
break;
}
printf("Intermediateresult:%d\n",result);
push(result);
```

```
}
}
printf("FinalResult=%d\n",pop());
return0;
}
```

Output:

3)L-Attributed Definition:

```
#include<stdio.h>
intE(intinherited)
{
intt=0;
printf("InheritedattributereceivedbyE:%d\n",inherited);
returninherited+t;
}
intmain()
{
inta=2;
intb=3;
intc=4;
intmultiplication;
intresult;
printf("Expression:a+b*c\n");
multiplication=b*c;
```

```
result=a+multiplication;
E(a);
printf("Inheritedattributepassedtochild:%d\n",a);
printf("b*c=%d\n",multiplication);
printf("FinalResult=%d\n",result);
return0;
}
```

Output:

4)Type Expression Equivalence:

```
#include<stdio.h>
#include<string.h>
inttypeEquivalent(chartype1[],chartype2[])
{
returnstrcmp(type1,type2)==0;
}
intmain()
{
chartype1[20],type2[20];
printf("Enterfirsttype expression:");
scanf("%s",type1);
printf("Entersecondtypeexpression:");
scanf("%s",type2);
if(typeEquivalent(type1,type2))
{
printf("TypesareEquivalent\n");
```

```
}
else
{
printf("TypesareNotEquivalent\n");
}
return0;
}
```

Output:

\

5)Semantic Type Checking:

```
#include<stdio.h>
#include<string.h>
intisValid(chartype1[],chartype2[],charop)
{
if(strchr(type1,'*')!=NULL||
strchr(type2,'*')!=NULL)
{
return0;
}
if((strcmp(type1,"int")==0||
strcmp(type1,"float")==0)&&
(strcmp(type2,"int")==0||
strcmp(type2,"float")==0))
{
if(op=='+'||op=='-'||
op=='*'||op=='/')
```

{

```
return1;
}
}
return0;
}
intmain()
{
chartype1[20],type2[20];
charop;
printf("Enterfirstoperandtype:");
scanf("%s",type1);
printf("Enteroperator(+,-,*,/):");
scanf("%c",&op);
printf("Entersecondoperandtype:");
scanf("%s",type2);
printf("\nExpression:%s%c%s\n",
type1,op,type2);
if(isValid(type1,type2,op))
{
printf("Validexpression\n");
}
else
{
printf("TypeError\n");
}
return0;
```

```
}
```

Output:

RUBRICS FOR CALCULATION

Criteria Weightage Level 4 – Exemplary Level 3 – Proficient Level 2 – Developing Level 1 – Beginning

Understanding ofConcepts 25 Demonstrates thorough understanding and effectivelyintegrates multiple concepts/technologies Good understanding with proper integration of mostconcepts Basic understanding; limitedorpartial integration Poor

understanding;

unable to

integrate

concepts

Experimental

Design 30

Well-planned,

systematic

implementationwith

optimalsolution

Correct

implementation

withminorissues

Basic

implementation

witherrorsor

inefficiencies

Incorrector

incomplete

implementation

UseofTools 20

Effectiveand

advanceduseof

tools/technologies

with proper

configuration

Appropriateuse oftoolswith correctexecution

Limitedor inconsistentuse oftools

Incorrector minimaluseof tools

Analysis of Results 15

Insightfulanalysis with accurate interpretationand validationofresults

Goodanalysis withreasonable interpretation

Basicanalysis withlimited insights

Poor orincorrect analysisof